

Developers Blog

Follow: 

Developers

Agent Memory: Why Your AI Has Amnesia and How to Fix It

February 17, 2026 | 16 minute read



Casius Lee

AI Developer Advocate

Key Takeaways

Today's AI agents forget everything between conversations. Every interaction starts from zero, with no recall of who you are or what you've discussed before.

Agent memory isn't about bigger context windows. It's about a persistent, evolving state that works across sessions.

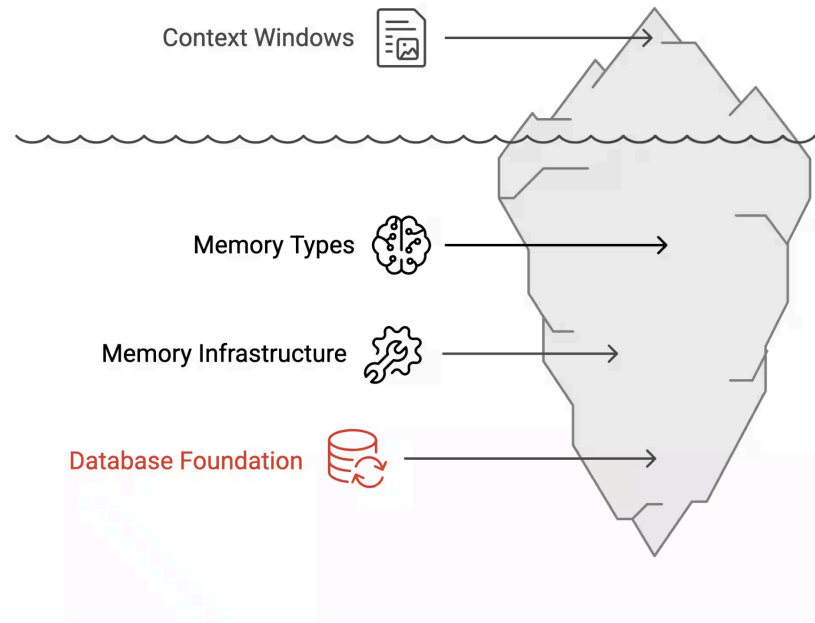
The field has converged on four memory types (working, procedural, semantic, episodic) that map directly to how human memory works.

Building agent memory at enterprise scale is fundamentally a database problem. You need vectors, graphs, relational data, and ACID transactions working together.



Agent Memory: What Lies Beneath the Context Window

Bigger prompts aren't the answer. Persistent memory is.



What Is Agent Memory and Why Does Your AI Agent Need It?

You've spent weeks building an AI customer service agent. It handles complaints, processes refunds, even cracks the occasional joke when the moment's right. A customer calls back the next day, and your agent has no idea who they are. The conversation from yesterday? Gone. The preference they mentioned twice last week? Never happened. Every single interaction starts from scratch.

This isn't a bug in your code. It's a fundamental design problem in how we build AI agents today.

LangChain put it well: *'Imagine if you had a coworker who never remembered what you told them, forcing you to keep repeating that information'*. In the coworker scenario, that's frustrating, and for AI applications, forgetfulness, that's a dealbreaker.

At Oracle, we've been deep in this problem as we continue to provide support to customers building AI applications. And here's what we've found: the solution isn't bigger context windows or more verbose prompts. It's a proper memory infrastructure. The kind that databases have been providing for decades.

Agent memory is the composition of system components and infrastructure layer that gives AI agents a persistent, evolving state across conversations and sessions. It enables agents to store, retrieve, update, and forget information over time: learning user preferences, retaining context from past interactions, and adapting behavior based on accumulated experience. Without it, every interaction starts from zero.

This article breaks down what agent memory actually is, how it works under the hood, the frameworks shaping the field, and guidance on how to build it for production. Whether you're prototyping your first agent or scaling one to thousands of users, this is the foundation you need to get right.



Why Bigger Context Windows Aren’t the Answer

The rapid expansion of context windows, now ranging from hundreds of thousands to millions of tokens, has created a convincing illusion across the industry: that with this much capacity available, the memory problem is effectively solved and retrieval-based mechanisms are behind us. That assumption is wrong.

The industry calls it ‘the illusion of memory’. Stuffing more tokens into a prompt isn’t memory. It’s a bigger Post-it note: more space to scribble on, but it still goes in the bin when the conversation ends. Memory means the notes survive. Here’s why that distinction matters:

Context windows degrade before they fill up. Most models break well before their advertised limits. A model claiming 200K tokens typically becomes unreliable around 130K, with sudden performance drops rather than gradual degradation.

There’s no sense of importance. Context windows treat every token equally. Your name gets the same weight as a throwaway comment from three weeks ago. There’s no prioritisation, no salience, no relevance filtering.

Nothing persists. Close the session and it’s all gone. Every conversation starts from zero.

The cost scales linearly. Maintaining full context across a long agent lifetime gets expensive fast. You’re paying per token, and most of those tokens are irrelevant noise.

Memory is not only about storing chat history or passing more tokens into the context window. It’s about building a persistent state stored in an external system, that evolves and informs every interaction the agent has, even weeks or months apart.

Another misconception to address early on is that RAG (retrieval augmented generation) is agent memory. **RAG brings external knowledge into the prompt at inference time.** It’s great for grounding responses with facts from documents. But RAG is fundamentally stateless. It has no awareness of previous interactions, user identity, or how the current query relates to past conversations. Memory brings continuity. Put simply: RAG helps an agent answer better. Memory helps it learn and adapt. You need both.

The Concept: A Mental Model for Agent Memory

Let me give you a framework that makes all of this click. It maps directly to how your own brain works.

In 2023, researchers at Princeton published the [CoALA framework](#) (Cognitive Architectures for Language Agents). It defines four types of memory, drawn from cognitive science and the [SOAR architecture](#) of the 1980s. Every major framework in the field builds on this taxonomy, and it answers a fundamental question: what options are available for adding memory to an AI agent?

Memory Type	Human Equivalent	What It Does in an Agent	Example Implementation
Working Memory	Your brain’s scratch pad: holding what you’re actively thinking about	Current conversation context, retrieved data, intermediate reasoning	Conversation buffers, sliding windows, rolling summaries, scratchpads
Procedural Memory	Muscle memory: knowing how to ride a bike without thinking	System prompts, agent code, decision logic	Prompt templates, tool definitions, agent configs
Semantic Memory	General knowledge: facts and concepts accumulated over your lifetime	User preferences, extracted facts, knowledge bases	Vector stores with similarity search
Episodic Memory	Autobiographical memory: recalling specific experiences from your past	Past action sequences, conversation logs, few-shot examples	Timestamped logs with metadata filtering

Think of it this way. When you’re in a meeting, your working memory holds what’s being discussed right now. Your procedural memory knows how to take notes and when to speak up. Your semantic memory reminds you that Sarah’s team



prefers Slack over email. Your episodic memory recalls that the last time you proposed this feature, the VP shut it down because of budget constraints.

An agent needs all four types working together. Most agents today only have working memory: whatever fits in the current context window. That's like trying to do your job using nothing but a whiteboard that gets wiped clean every evening.

Lilian Weng's influential formula captures the big picture simply:

Agent = LLM + Memory + Planning + Tool Use.

Her short-term memory maps to CoALA's working memory. Her long-term memory encompasses the other three types.

LangChain adds a practical layer with two approaches to memory updates:

- Hot path memory:** the agent explicitly decides to remember something before responding. This is what ChatGPT does. It adds latency but ensures immediate memory updates.
- Background memory:** a separate process extracts and stores memories during or after the conversation. No latency hit, but memories aren't available straight away.

The key insight: memory is application-specific. What a coding agent remembers about a user is very different from what a research agent might store.

Letta (formerly MemGPT) takes a different angle entirely, borrowing from operating systems. Treat the context window like RAM and external storage like a disk. The agent pages data between these tiers, creating a 'virtual context' that feels unlimited. The agent manages its own memory using tools: it decides what to remember, what to update, and what to archive.

The distinction between programmatic memory (developer decides what to store) and agentic memory (the agent itself decides) matters. The field is moving towards the latter. Agents that manage their own memory adapt to individual users without requiring developer intervention for each new use case. The decision as to which memory operations are programmatic and agent triggered isn't always as clear cut, and we've seen various approaches work well in certain use cases and domains. In a future post, we will go into the common patterns and design principles of memory engineering.

Referring back to the customer service agent from the start of this article. Customer service is the most common use case for agents in production (26.5% of deployments, per LangChain's 2025 industry survey), and it demands all four memory types working together. Episodic memory recalls past tickets and interactions. Semantic memory stores customer preferences and account details. Working memory tracks the live conversation. Procedural memory encodes resolution workflows and escalation rules. All four memory types enable the chatbot to perform well on continuous tasks and adapt to new information.



The Landscape: Frameworks and Open-Source Libraries

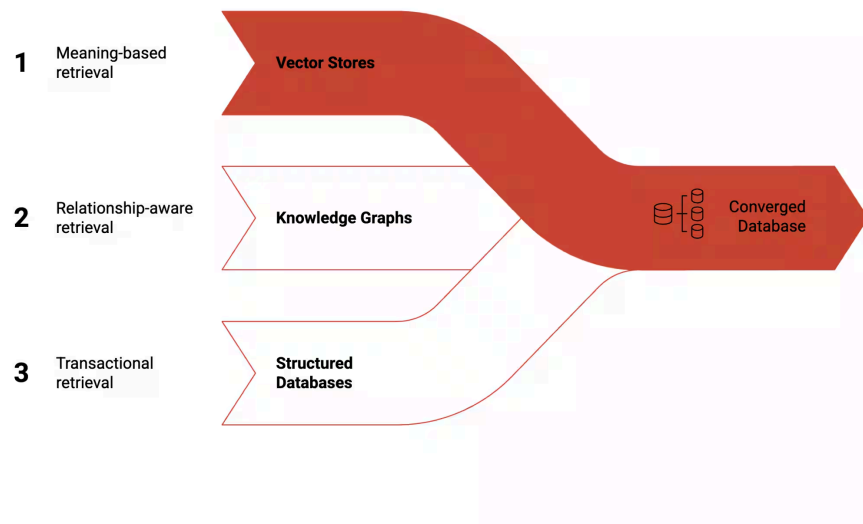
What are the commonly used libraries and open-source projects for agent memory? The ecosystem has matured quickly. Here are the projects shaping how developers build agent memory today.

Project	What It Does	Open Source
LangChain / LangMem / LangGraph	Agent orchestration with built-in memory abstractions. Hot path and background memory. LangMem SDK handles extraction and consolidation.	Yes
Letta (MemGPT)	Stateful agent platform with OS-inspired memory hierarchy. Agents self-edit their own memory via tool calls.	Yes
Zep / Graphiti	Temporal knowledge graphs for relationship-aware memory. Bi-temporal modelling with sub-200ms retrieval.	Yes (Graphiti)
Mem0	Self-improving memory layer with vector and graph architecture. Automatic memory extraction and conflict resolution.	Yes
langchain-oracledb	Official LangChain integration for Oracle Database. Vector stores, hybrid search, and embeddings with enterprise-grade security.	Yes

The orchestration library matters, but at scale, the storage backend matters more. Most of these frameworks are database-agnostic by design. The question isn't which framework to use. It's what database sits underneath it.

The Three Pillars of AI Memory

Three paradigms. One database. Zero trade-offs.



The Deep Dive: How Agent Memory Actually Works

What are the common storage options for agent memory? Production systems today use three paradigms working together. You need to understand all three.

Vector stores for semantic memory

This is the most common approach. You take text, convert it to embeddings (typically 128 to 2,048 dimensions depending on embedding model utilised), and store them in a vector database. Retrieval works through vector search, against vectors that are indexed using HNSW (hierarchical navigable small world); typically we find the memories (embeddings in database) that are semantically closest to the current query.

It's fast and simple but limited. Vector search captures semantic similarity well, yet misses structural relationships.

Knowledge graphs for relationship memory

Vector search can tell you that a user mentioned coffee. But it can't tell you that they prefer a specific shop, ordered last Tuesday, and always get oat milk. That chain of connections (person, preference, place, time, detail) is a graph problem.

Knowledge graphs store facts as entities and relationships, with edges capturing how they connect. Add bi-temporal modelling (tracking both when events happened and when the system learned about them) and you can ask not just 'what do we know?' but 'what did we know at any point in time?'

Frameworks like Zep's Graphiti implement this pattern, [achieving 94.8% accuracy](#) on the Deep Memory Retrieval benchmark. Oracle Database supports property graphs natively through SQL/PGQ, so graph queries run inside the same engine as your vector search and relational data.

Structured databases for factual memory

Relational databases store the structured data: user profiles, access controls, session metadata, audit logs. As [Cognee](#) puts it: "Vectors deliver high-recall semantic candidates (what feels similar), while graphs provide the structure to trace relationships across entities and time (how things relate)." Relational tables anchor both with the transactional guarantees that production systems demand.

Why does a converged database change the equation?

Most teams stitch this together with separate databases: Pinecone for vectors, Neo4j for graphs, Postgres for relational data. Three security models, three failure modes, no shared transaction boundaries. If one write fails, your agent's memory is in an inconsistent state.

Oracle's converged database runs all three paradigms natively inside a single engine:

AI Vector Search for embedding storage and similarity retrieval

SQL/PGQ for property graph queries across entity relationships

Relational tables for structured data, metadata, and audit trails

JSON Document Store for flexible, schema-free memory objects

All four share the same ACID transaction boundary and the same security model. Row-level security, encryption, and access controls apply uniformly across every data type. One engine, one transaction, one security policy: the three paradigms above become three views of the same underlying data.

The four memory operations

Every memory system runs on four core operations:

1. **ADD:** Store a completely new fact
2. **UPDATE:** Modify an existing memory when new information complements or corrects it
3. **DELETE:** Remove a memory when new information contradicts it
4. **SKIP:** Do nothing when information is a repeat or irrelevant

Modern memory systems delegate these decisions to the LLM itself rather than using brittle if/else logic. The extraction phase ingests context sources (the latest exchange, a rolling summary, recent messages) and uses the LLM to extract candidate memories. The update phase compares each new fact against the most similar entries in the vector database, using conflict detection to determine whether to add, merge, update, or skip.

Retrieval: how agents recall

Due to the heterogeneous nature of data that agents encounter, production systems combine multiple retrieval approaches:

Semantic search: vector similarity (cosine distance) for meaning-based matching

Temporal search: bi-temporal models enable point-in-time queries ("What did the user prefer last March?")

Graph traversal: multi-hop queries across knowledge graph edges for complex reasoning

Hybrid retrieval: combining keyword (full-text) and semantic (vector) search in a single query, which is critical for retrieving specific facts like names, dates, or project codes alongside conceptually related memories

Forgetting: the underrated operation

Effective forgetting can be implemented with decay functions applied to vector relevance scores: by analysing the results of vector search, old and unreferenced embeddings naturally fade from the agent's attention, imitating biological human memory decay patterns. In a database, this is straightforward. A recency-weighted scoring function multiplies semantic similarity by an exponential decay factor based on time since last access. The result: memories that haven't been recalled recently lose salience gradually, just like human recall.

Some systems take a different approach entirely. Old facts are invalidated but never discarded, preserving historical accuracy for audit trails. The right strategy depends on your use case, but both are fundamentally database operations.

The Enterprise Reality: What Changes at Scale



Here's where the gap between demo and production becomes a chasm.

[KPMG's Pulse Survey](#) of 130 C-suite leaders (all at companies with over \$1B revenue) found that 65% cite agentic system complexity as the top barrier for two consecutive quarters. Agent deployment has more than doubled, from 11% in Q1 2025 to 26% in Q4 2025, but that still means three quarters of large enterprises haven't deployed. [McKinsey](#) puts it even more starkly: only 1% of leaders describe their companies as 'mature' in AI deployment.

The problems that surface at scale are database problems. They've been database problems all along.



Security and isolation. Memory must be scoped per user, per team, per organisation. Memory poisoning is a real attack vector: adversaries can inject malicious information into an agent's memory to corrupt future decision-making. You need row-level security, not just namespace-level isolation.

Multi-tenancy. Agents serving multiple organisations need complete data isolation. Most vector-only databases offer namespace-level separation. That's not the same as the row-level security that regulated industries require. Oracle's native PDB/CDB architecture provides inherent multi-tenant isolation.

Compliance is getting complex. GDPR's right to be forgotten applies to explicit agent memory stores. But the EU AI Act (fully applicable from August 2026) requires 10-year audit trails for high-risk AI systems. Think about that tension: you need to delete personal data on request while maintaining a decade of audit history. That requires architectural sophistication that most startups are only beginning to address.

ACID transactions matter. Agent memory operations often touch multiple data types simultaneously. Updating a vector embedding, modifying a graph relationship, and changing relational metadata must all succeed or all fail. Without atomicity, partial memory updates leave your agent in an inconsistent state.

These aren't theoretical concerns. They're the reasons three quarters of enterprises are still stuck at the pilot stage.

The Implementation: Building Agent Memory with LangChain and Oracle

Let's get practical. We'll use LangChain as our orchestration framework and Oracle Database as the memory backend, using the langchain-oracledb package. Here's how quickly you can go from zero to a working memory system.

[Copy code snippet](#)

```
pip install langchain-oracledb oracledb langchain-core
```

Connect and create a vector store

This is all it takes to set up a production-ready vector store backed by Oracle:

[Copy code snippet](#)

```
import oracledb
from langchain_oracledb.vectorstores import OracleVS

# Create a connection pool (production-ready)
pool = oracledb.create_pool(
    user="agent_user", password="password",
    dsn="hostname:port/service",
    min=2, max=10, increment=1
)

# Initialise vector store for semantic memory
semantic_memory = OracleVS(
    client=pool.acquire(),
    embedding_function=embeddings, # any LangChain-compatible embeddings
    table_name="AGENT_SEMANTIC_MEMORY",
    distance_strategy=DistanceStrategy.COSINE,
)
```

That's your semantic memory store. Oracle handles the vector indexing, ACID transactions, and security natively. No separate vector database needed.

Store and retrieve a memory

The core pattern is simple: write memories with metadata, retrieve them with similarity search.

[Copy code snippet](#)

```
# Store a memory
semantic_memory.add_texts(
    texts=["User prefers dark mode and concise responses."],
    metadatas=[{"user_id": "user_123", "memory_type": "preference"}]
)

# Retrieve relevant memories
results = semantic_memory.similarity_search(
    "What are this user's preferences?",
    k=5,
    filter={"user_id": "user_123"}
)
for doc in results:
    print(doc.page_content)
```

From here, you can create separate vector stores for each memory type (semantic, episodic, procedural) under the same Oracle instance, all sharing the same security policies and transaction guarantees.

Go deeper: the full memory engineering notebook

The snippets above show the building blocks, but a production agent memory system needs considerably more. We've published a complete, runnable notebook in the [Oracle AI Developer Hub](#) that implements the full architecture discussed in this post. This notebook builds a complete Memory Manager with **six distinct memory types**, each backed by Oracle:

Memory Type	Purpose	Storage
Conversational	Chat history per thread	SQL Table
Knowledge Base	Searchable documents and facts	SQL Table + Vector Enabled
Workflow	Learned action patterns	SQL Table + Vector Enabled

Toolbox	Dynamic tool definitions with semantic retrieval	SQL Table + Vector Enabled
Entity	People, places, systems extracted from context	SQL Table + Vector Enabled
Summary	Compressed context for long conversations	SQL Table + Vector Enabled

It also covers **context engineering** (monitoring context window usage, auto-summarisation at thresholds, just-in-time retrieval), **semantic tool discovery** (scaling to hundreds of tools while only passing the relevant ones to the LLM), and a **complete agent loop** that ties everything together.

Run the notebook → [oracle-devrel/oracle-ai-developer-hub](#)

The Perspective: Where This Is Heading

Here's what I think is coming, and where I'm still working things out.

Sleep-time computation will change the game. The idea is simple: agents that 'think' during idle time (reorganising, consolidating, refining their memories) perform better and cost less at query time. [OpenAI's internal data](#) agent already runs this pattern in production. Their engineering team describes a daily offline pipeline that aggregates table usage, human annotations, and code-derived enrichment into a single normalised representation, then converts it into embeddings for retrieval. At query time, the agent pulls only the most relevant context rather than scanning raw metadata.

[Letta's](#) research puts numbers to it: agents using this approach achieve 18% accuracy gains and 2.5x cost reduction per query. We're going to see a clear separation between 'thinking agents' that run in the background and 'serving agents' that handle real-time interactions. That's a pattern databases have supported forever: batch processing alongside real-time queries.

Memory will extend naive RAG implementations. The spectrum is already shifting: traditional RAG to agentic RAG to full memory systems. VentureBeat predicts that contextual memory will surpass RAG for agentic AI in 2026. I think that's right. RAG retrieves documents. Memory understands context. The agents that win will do both, but memory will be the differentiator.

The convergent database will become non-negotiable. Agent memory needs vectors, graphs, relational data, and temporal context working together. Stitching together separate databases for each type creates brittle systems with security gaps and consistency problems. I'm still figuring out exactly how fast this consolidation will happen, but the direction is clear.

One open question remains, and that is the pace at which enterprises will transition from pilot to production deployment. At the moment the technology is at a clear stage of maturity and architectural design patterns are proven and battle tested. On the other hand, organisational readiness, encompassing governance, infrastructure modernisation, and cross-functional alignment, is a fundamentally different challenge.

What is clear: agent memory is, at its foundation, a database problem. And building databases for mission-critical workloads is what Oracle has been doing for nearly five decades.

Frequently Asked Questions

What are the main types of agent memory used in AI systems?

The field has converged on four types, drawn from cognitive science: **working memory** (current conversation context), **procedural memory** (system prompts and decision logic), **semantic memory** (accumulated facts and user preferences), and **episodic memory** (past interaction logs and experiences). Every major framework builds on this taxonomy, first formalised in the CoALA framework from Princeton in 2023.

What options are available for adding memory to an AI agent?

Two broad approaches exist. **Programmatic memory** is where the developer defines what gets stored and retrieved. **Agentic memory** is where the agent itself decides what to remember, update, and forget using tool calls. Frameworks like Letta (formerly MemGPT) and LangChain's LangMem SDK support both patterns. The field is moving towards agentic memory, where agents manage their own state without developer intervention for each new use case.



What are common agent memory storage options?

Production systems typically combine three paradigms: **vector stores** for meaning-based retrieval (storing embeddings and querying by cosine similarity), **knowledge graphs** for relationship-aware retrieval (entities, edges, and bi-temporal modelling), and **structured relational databases** for transactional data like user profiles, access controls, and audit logs. Most teams stitch these together with separate databases, though converged databases like Oracle can run all three natively in a single engine.

What techniques allow AI agents to forget or selectively erase memory?

The most common approach uses **decay functions** applied to vector relevance scores: a recency-weighted scoring function multiplies semantic similarity by an exponential decay factor based on time since last access. Memories that haven't been recalled recently lose salience gradually, mimicking biological memory decay. An alternative approach **invalidates** old facts without discarding them, preserving historical accuracy for audit trails while removing them from active retrieval.

What are the differences between short-term and long-term agent memory?

Short-term memory (also called working memory) is the current context window: whatever the agent is actively reasoning about in this conversation. It's fast but volatile; close the session and it's gone. **Long-term memory** encompasses everything that persists across sessions: semantic memory (facts and preferences), episodic memory (past interactions), and procedural memory (learned behaviours and decision logic). Long-term memory requires external storage and retrieval infrastructure.

What are commonly used libraries for agent memory?

The ecosystem includes **LangChain/LangMem** (hot path and background memory with extraction and consolidation), **Letta/MemGPT** (OS-inspired memory hierarchy where agents self-edit memory via tool calls), **Zep/Graphiti** (temporal knowledge graphs with sub-200ms retrieval), **Mem0** (self-improving memory with automatic conflict resolution), and **langchain-oracledb** (Oracle Database integration for vector stores, hybrid search, and embeddings with enterprise-grade security).

How do I store and query vector embeddings?

The core pattern is straightforward: convert text into embeddings (typically 128 to 2,048 dimensions), store them in a vector-capable database, and retrieve them using cosine similarity search. With langchain-oracledb and Oracle Database, you initialise a vector store, add texts with metadata (such as user ID and memory type), then query with `similarity_search()` filtered by metadata. Oracle handles vector indexing, ACID transactions, and security natively.

Which databases offer vector search capabilities for enterprises?

Several databases now support vector search, but enterprise requirements go beyond basic similarity queries. You need ACID transactions, row-level security, multi-tenancy, and compliance features alongside your vector operations. Oracle Database provides native **AI Vector Search** within its converged architecture, meaning vector queries run in the same engine as relational tables, property graphs (SQL/PGQ), and JSON document stores, all sharing a single transaction boundary and security model.



Casius Lee
AI Developer Advocate

[← Previous post](#)

Build a Scalable Multi Agent RAG system with A2A Protocol and LangChain

[Nacho Martinez](#) | 2 minute read

[Next post →](#)

Using Functions with API Gateway

[Phill Denness](#) | 2 minute read

Resources for

About
Careers
Developers
Investors
Partners
Startups

Why Oracle

Analyst Reports
Best CRM
Cloud Economics
Corporate Responsibility
Security Practices

Learn

What is Customer Service?
What is ERP?
What is Marketing Automation?
What is Procurement?
What is Talent Management?
What is VM?

What's New

Try Oracle Cloud Free Tier
Oracle Sustainability
Oracle COVID-19 Response
Oracle and SailGP
Oracle and Premier League
Oracle and Red Bull Racing
Honda

Contact Us

US Sales 1.800.633.0738
How can we help?
Subscribe to Oracle Content
Try Oracle Cloud Free Tier
Events
News

